RFCS — Real-Time Fact Checking System

Complete Master Analysis Document

PART 1 — FINALIZED PIPELINE (ALL CHANGES INCORPORATED)

What was removed

- **Claim Type checkpoint** — dropped. Checkability + Context together already provide all the routing signal claim type would have added. No loss to pipeline quality.

Final pipeline stages in order

```
INPUT (Text / Image / PDF)
        │
        ▼
[STAGE 0]  Language Detection          fastText lid.176.bin — off-shelf, no
training
        │
        ▼
[STAGE 1]  Claim Normalization          text cleaning, encoding fix, NER-based
query gen
        │
        ├── IMAGE PATH: OCR first (Tesseract primary / EasyOCR fallback) →
normalized text → Stage 2
        └── PDF PATH:   section extraction → claim list → each claim enters Stage
2 independently
        │
        ▼
[STAGE 2]  Checkability Checkpoint      TRAINED MODEL (EN + Indic) — existing v5
checkpoints
        │
        ├── UNCHECKABLE → return reason (opinion / question / vague / future / no
referent)
        └── CHECKABLE  → continue
        │
        ▼
[STAGE 3]  Context / Domain Checkpoint  TRAINED MODEL (EN + Indic) — existing
checkpoints
           14 labels: SCIENCE · HEALTH · TECHNOLOGY · HISTORY ·
POLITICS_GOVERNMENT ·
```

```
           ECONOMICS_BUSINESS · GEOGRAPHY · SPACE_ASTRONOMY · ENVIRONMENT_CLIMATE ·
           SOCIETY_CULTURE · LAW_CRIME · SPORTS · ENTERTAINMENT · GENERAL_FACTUAL
        │
        ▼
[STAGE 4]  Domain Routing               CONFIG-BASED (no model) — domain_routing.json
           Context label → prioritized API/search/scrape sources
           Example: SPACE → NASA API first · HEALTH → OpenFDA + WHO · POLITICS_INDIA → PIB + PRS
        │
        ▼
[STAGE 5]  Evidence Gathering           3 source tiers in parallel
           5A: Structured APIs          NASA · OpenFDA · PIB · data.gov.in · PRS India · NewsAPI
           5B: Web Search               Tavily (primary) · SerpAPI · DuckDuckGo (fallback)
           5C: Scraping                 Trafilatura (primary) · Playwright (JS fallback) · BS4
           ↻  Loop-back if quality low  max 2 retries · unique queries only · circuit breaker
        │
        ▼
[STAGE 6]  Evidence Selection + MMR     ← MMR lives HERE, not in query generation
           Dedup by URL · domain diversity cap (max 3 per root domain)
           MMR re-ranks: balance relevance score vs source diversity
           Output: top 10 diverse candidate passages
        │
        ▼
[STAGE 7]  Relevance Checkpoint         TWO-STAGE INTERNAL PATTERN
           7A: Bi-encoder               multilingual-e5-small (off-shelf) — filter 100 → top 20
           7B: Cross-encoder            FINE-TUNED MODEL (EN + Indic) — rank 20 → top 5–8
           Filter: drop if RS < 0.30
        │
        ▼
[STAGE 8]  Stance Checkpoint            FINE-TUNED NLI MODEL (EN + Indic)
           Input: (claim, evidence_passage) pairs
           Output: [P_support, P_refute, P_neutral] per evidence
           Weighted by: RS × CW × TW (credibility + temporal weights)
           Aggregate across all evidence → agg_sup / agg_ref / agg_neu
        │
        ▼
```

```
[STAGE 9]  LLM Verifier                  Groq API — llama-3.3-70b-versatile
           Triggered ONLY when: verdict = NEUTRAL  OR  max confidence < 0.55
           Policy: neutral_only · max 3 evidence items passed · timeout 20s
        |
        ▼
[STAGE 10] Verdict Engine
           argmax(agg_sup, agg_ref, agg_neu) → raw label
           Apply confidence label mapping:
             SUPPORT ≥ 0.75 → Verified       0.55-0.75 → Likely True    < 0.55 →
LLM verifier
             REFUTE  ≥ 0.75 → False           0.55-0.75 → Likely False   < 0.55 →
LLM verifier
             NEUTRAL (any)  → LLM verifier always
           Conflict flag: if agg_sup > 0.30 AND agg_ref > 0.30 simultaneously
           Output: verdict label · confidence · conflict flag · top evidence chain
· explanation
```

PART 2 — IMAGE PIPELINE (DISTINCT FROM TEXT)

```
Image Input (JPG / PNG / WEBP)
        |
        ▼
Image Quality Check
  - Detect if image contains text (basic blur/contrast check)
  - If no text detected → return "No checkable text found in image"
  - If text detected → continue
        |
        ▼
OCR Stage (primary: Tesseract  |  fallback: EasyOCR)
  Tesseract:   fast, high accuracy on clean screenshots
               langs: eng+tel+hin+tam+kan+mal
  EasyOCR:     slower, better on curved/noisy/handwritten text
               fallback triggered when Tesseract confidence < 0.70
        |
        ▼
OCR Post-processing
  - Remove OCR artifacts (garbled chars, broken words)
  - Encoding fix (legacy script encodings → UTF-8)
  - Stitch multi-line text into coherent sentences
        |
        ▼
```

```
Text extraction types — SCOPED FOR V1 (supported):
  ✓ Screenshots of social media posts
  ✓ Screenshots of news headlines
  ✓ WhatsApp / Telegram forwarded text screenshots
  ✓ Printed text photos with clean background

  OUT OF SCOPE FOR V1 (not supported, return graceful error):
  × Charts / graphs / bar charts
  × Natural photos with no text
  × Handwritten text
  × Memes with overlaid stylized text (very low OCR accuracy)
        |
        ▼
Extracted text → enters main pipeline at Stage 0 (Language Detection)
```

Why EasyOCR as fallback, not primary: Tesseract is ~10x faster and adequate for clean screenshots. EasyOCR is more robust but adds 2–4 seconds per image. For academic project, speed matters less — but structuring Tesseract as primary keeps the common case fast.

PART 3 — PDF PIPELINE (OPTIONAL — BUILD AFTER TEXT + IMAGE STABLE)

```
PDF Input
        |
        ▼
Page count check
  ≤ 5 pages  → show "time intensive" warning, proceed
  > 5 pages  → hard stop, return error asking user to select pages
        |
        ▼
Section/Subsection Extraction
  Tool: pdfplumber (better than PyPDF2 for layout-aware extraction)
  Extract: headings, paragraphs, tables as text, captions
  Output: structured dict {page_num: {section_title: [paragraph_texts]}}
        |
        ▼
Claim Mining per Section
  Each paragraph → run Checkability checkpoint
  Collect all CHECKABLE paragraphs as candidate claims
  Deduplicate near-identical claims (cosine similarity > 0.92)
        |
        ▼
Batch Push to Main Pipeline
```

```
  Each checkable claim enters Stage 3 (Context) independently
  (Checkability already done during mining step)
  Run pipeline stages 3-10 per claim
  Collect all verdicts
        |
        ▼
PDF-level Verdict Report
  Per-page summary:
    PAGE 1: N claims analyzed · X supported · Y refuted · Z neutral
    PAGE 2: ...
  Per-section summary:
    Section "Introduction": 3 claims · verdict summary
  Document-level verdict:
    Based on proportion of supported/refuted/neutral across all claims
    Highly Reliable / Mostly Reliable / Mixed / Mostly Misleading / Highly
Misleading
  Credibility report:
    Total claims found · checkable count · analyzed count · stance distribution
```

Key difference from text pipeline: PDF pipeline is a wrapper that mines claims from a document and batches them through the same single-claim pipeline. No new models are needed. The verdict aggregation at document level is a simple proportion calculation, not a new model.

PART 4 — MODEL DECISIONS PER CHECKPOINT

Decision table

Checkpoint	EN Model	Indic Model	Status	Fine-tune needed?
Language Detection	fastText lid.176.bin	same (universal)	Off-shelf	NO
NER (query gen)	spaCy en_core_web_trf	AI4Bharat IndicNER	Off-shelf	NO
Checkability	existing v5_public_large	existing v5_public_large_multilingual	Existing	EVALUATE FIRST
Context/Domain	existing context_model_final	existing MuRIL or indic-bert	Existing	EVALUATE FIRST

Checkpoint	EN Model	Indic Model	Status	Fine-tune needed?
Relevance bi-encoder	multilingual-e5-small (shared)	same model, shared	Off-shelf	NO
Relevance cross-encoder	msmarco-MiniLM-L6 → fine-tune FEVER	mmarco-mMiniLMv2 → fine-tune FEVER+IndicQA	Fine-tune	YES
Stance	DeBERTa-v3-base → MNLI→FEVER→VitaminC	mDeBERTa-v3-base → IndicNLI+translated FEVER	Fine-tune	YES
LLM Verifier	llama-3.3-70b-versatile via Groq	same (API handles multilingual)	API	NO

Models that need fine-tuning: exactly 4

1. **Relevance cross-encoder EN** — start from `cross-encoder/ms-marco-MiniLM-L-6-v2`, fine-tune on FEVER evidence pairs
2. **Relevance cross-encoder Indic** — start from `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`, fine-tune on FEVER + IndicQA
3. **Stance EN** — start from `microsoft/deberta-v3-base`, staged: MNLI → FEVER → VitaminC
4. **Stance Indic** — start from `microsoft/mdeberta-v3-base`, train on IndicNLI + translated FEVER

Models that are off-shelf (no training): 3

- fastText lid.176.bin
- spaCy en_core_web_trf + AI4Bharat IndicNER
- intfloat/multilingual-e5-small (bi-encoder — shared for both EN and Indic)

Models to evaluate before deciding: 2

- Existing checkability v5 checkpoints → run on CLEF 2022 test → if Macro-F1 > 0.82, keep as-is
- Existing context_model_final → run on DBpedia 14 test → if accuracy > 0.83, keep as-is

---

PART 5 — DATASETS

Stance EN datasets (in training order)

Dataset	Size	Purpose	Download
MultiNLI (MNLI)	392,702 pairs	General NLI pre-training warmup	HuggingFace: `multi_nli`
FEVER v1.0	145,449 train / 19,998 dev / 19,998 test	Core fact-check NLI	HuggingFace: `fever` `v1.0`
VitaminC	370,459 pairs	Contrastive NLI — hard negatives, prevents surface-match bias	HuggingFace: `tals/vitaminc`

Training order: Pre-train on MNLI (1 epoch) → fine-tune on FEVER (3 epochs) → fine-tune on VitaminC (2 epochs). This staged approach consistently outperforms joint training.

Stance Indic datasets

Dataset	Languages	Size	Download
IndicNLI (AI4Bharat)	11 Indian languages incl. te/kn/ml/ta/hi	~50k per language	`ai4bharat/IndicNLI`
IndicXNLI	Hindi + 5 others	5,000 test per language	HuggingFace: `Divyanshu/indicxnli`
XNLI	Hindi + 14 others	5,000 test per language	HuggingFace: `xnli`
Translated FEVER → Hindi	Hindi	~50,000 (generate via IndicTrans2)	Generate yourself
Translated VitaminC → Hindi	Hindi	~50,000 (generate via IndicTrans2)	Generate yourself

For Telugu / Kannada / Malayalam specifically: IndicNLI is your only significant source. Cross-lingual transfer from Hindi + Tamil covers the gap, but expect 5–8% lower F1 on these three languages vs Hindi.

Relevance EN datasets

Dataset	Size	Purpose	Download
FEVER evidence retrieval	185,455 claim-evidence pairs	Primary fine-tune source	HuggingFace: `fever`
MS-MARCO passage (cross-encoder format)	~400k pairs	General relevance pretraining	HuggingFace: `ms_marco`

Relevance Indic datasets

Dataset	Languages	Size	Download
IndicQA (AI4Bharat)	Hindi · Bengali · Tamil · Telugu · Malayalam	~6,000 QA pairs	`ai4bharat/IndicQA`
Mr.TyDi (Hindi)	Hindi	~5,000 queries	HuggingFace: `mr_tydi`

Checkability evaluation datasets (not training — eval only)

Dataset	Purpose	Download
CLEF CheckThat! 2022 Task 1A EN test	Evaluate existing EN checkability model	checkthat.gitlab.io
CLEF CheckThat! 2023 Task 1A EN test	Additional eval data	checkthat.gitlab.io
ClaimBuster scored dataset	Supplementary eval	claimbuster.uta.edu

Context/Domain evaluation datasets

Dataset	Purpose	Download
DBpedia 14 test split	Evaluate existing EN context model	HuggingFace: `dbpedia_14`
IndicNLP News Classification test	Evaluate Indic context model	`ai4bharat/IndicNLPSuite`

PART 6 — TRAINING HARDWARE PLAN

Resource summary

Platform	GPU	VRAM	Session limit	Weekly quota
RTX 4050 (local)	RTX 4050	8 GB	Unlimited	Unlimited
Kaggle Account 1	T4 × 2	32 GB	12 hr/session	30 hr/week
Kaggle Account 2	T4 × 2	32 GB	12 hr/session	30 hr/week
Colab Account 1	T4 or L4	15–22 GB	~4 hr/day	~28 hr/week
Colab Account 2	T4 or L4	15–22 GB	~4 hr/day	~28 hr/week

Training assignment

RTX 4050 — evaluation only + small utilities

All eval scripts run here. The 4050 can also handle:

- Fine-tuning MiniLM-based models (< 80M params) if needed
- Running inference on all checkpoints during pipeline testing
- Data preprocessing, tokenization, dataset preparation

Do NOT attempt DeBERTa-v3-base or mDeBERTa on the 4050 — 8GB VRAM is too tight for training, but fine for inference with batch size 1.

Kaggle Account 1 — Stance EN

Model: `microsoft/deberta-v3-base`
Training: staged MNLI → FEVER → VitaminC

Session plan (each session = 1 Kaggle run, save checkpoint to Kaggle Dataset between runs):

- Session 1 (up to 12 hr): MNLI warmup — 1 epoch, batch 8 + grad_accum 4 = effective batch 32
- Session 2 (up to 12 hr): FEVER fine-tune — 3 epochs, same batch config
- Session 3 (up to 12 hr): VitaminC fine-tune — 2 epochs

Save checkpoint to Kaggle Dataset output after each session. Load from Dataset in next session.

Config:

```
batch_size: 8
gradient_accumulation_steps: 4
learning_rate: 2e-5
max_length: 512
fp16: true
eval_steps: 500
save_steps: 500
```

Kaggle Account 2 — Stance Indic

Model: `microsoft/mdeberta-v3-base`
Training: IndicNLI + translated FEVER

Session plan:

- Session 1 (up to 12 hr): IndicNLI training — 3 epochs across all available languages
- Session 2 (up to 12 hr): Translated FEVER Hindi fine-tune — 2 epochs
- Session 3 (up to 8 hr): Evaluation across all 5 languages on IndicNLI test splits

Config:

```
batch_size: 8
gradient_accumulation_steps: 4
learning_rate: 2e-5
max_length: 256  (shorter — multilingual tokenizer is more efficient)
fp16: true
```

Colab Account 1 — Relevance cross-encoder EN

Model: `cross-encoder/ms-marco-MiniLM-L-6-v2` (22M params — fits easily)
Training: FEVER evidence pairs + MS-MARCO pairs

This is a short training job — MiniLM is small. Expect 2–4 Colab sessions total.

- Session 1 (4 hr): MS-MARCO pretraining — 1 epoch
- Session 2 (4 hr): FEVER fine-tune — 3 epochs
- Session 3 (2 hr): Evaluation — Recall@5, MRR@10 on FEVER dev

Save to Google Drive between sessions.

Config:

```
batch_size: 32  (small model, can use larger batch)
learning_rate: 7e-6
max_length: 512
fp16: true
```

Colab Account 2 — Relevance cross-encoder Indic

Model: `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`
Training: FEVER + IndicQA cross-encoder triplets

- Session 1 (4 hr): FEVER multilingual fine-tune
- Session 2 (4 hr): IndicQA fine-tune
- Session 3 (2 hr): Evaluation on IndicQA test + FEVER

Save to Google Drive between sessions.

Session checkpoint saving — critical pattern

Since Kaggle cuts off at 12 hr and Colab at ~4 hr, never rely on a single session completing training. Use this pattern in every notebook:

```python
# At end of each epoch, save to persistent storage
trainer.save_model(f"./checkpoint_epoch_{epoch}")

# For Kaggle: copy to /kaggle/working/ which persists as output dataset
# For Colab: copy to /content/drive/MyDrive/rfcs_checkpoints/
```

In the next session, load the checkpoint and continue:

```python
model = AutoModelForSequenceClassification.from_pretrained("./checkpoint_epoch_N")
```

Always track which epoch you're resuming from in a `training_log.json` file alongside the checkpoint.

PART 7 — EVALUATION PLAN PER CHECKPOINT

Checkability (existing models — evaluate before deciding to retrain)

Datasets: CLEF CheckThat! 2022 Task 1A EN test, CLEF CheckThat! 2023 Task 1A EN test
Script: `training/local_4050/eval_checkability.py`

Metrics to report:

- Precision, Recall, F1 per class (CHECKABLE / UNCHECKABLE)
- Macro-F1 (primary metric)
- MAP (Mean Average Precision — official CLEF metric, based on confidence ranking)
- Confusion matrix

Decision rule:

- Macro-F1 > 0.82 → keep existing checkpoint, no retraining
- Macro-F1 0.75–0.82 → acceptable for academic, note limitation in paper
- Macro-F1 < 0.75 → retrain on DeBERTa-v3-base using CLEF 2019–2023 combined

Context/Domain (existing models — evaluate before deciding to retrain)

Datasets: DBpedia 14 test (EN), IndicNLP News Classification test (Indic)
Script: `training/local_4050/eval_context.py`

Metrics to report:

- Accuracy (primary for single-label)
- Macro-F1
- Confusion matrix 14×14 — look specifically for POLITICS↔SOCIETY confusion and SCIENCE↔HEALTH confusion

Decision rule:

- Accuracy > 0.83 on DBpedia → keep existing
- Accuracy 0.78–0.83 → acceptable
- Accuracy < 0.78 → retrain

Relevance cross-encoder (after fine-tuning)

Dataset: FEVER evidence retrieval dev split

Script: `training/local_4050/eval_relevance.py`

Metrics to report:

- Recall@1, Recall@5, Recall@10 (what % of time is gold evidence in top-N)
- MRR@10 (Mean Reciprocal Rank)
- Precision@1

Target: Recall@5 > 0.85

Also do manual spot-check: 50 random claims from your test set, scrape evidence yourself, check if model ranks the most relevant passage at top.

Stance (after fine-tuning)

Datasets: FEVER v1.0 labelled_dev (EN), IndicNLI per-language test (Indic)
Script: `training/local_4050/eval_stance.py`

Metrics to report:

- 3-way accuracy (primary)
- Macro-F1
- Per-class F1: SUPPORT / REFUTE / NOT ENOUGH INFO separately
- Confusion matrix 3×3

What to look for in confusion matrix:

- Over-prediction of NEUTRAL/NEI: model plays it safe. If > 50% of all predictions are NEI on a balanced test set, something is wrong — confidence is deflated.
- SUPPORT ↔ NEUTRAL confusion: model can't distinguish partial support from full support.
- REFUTE being stronger than SUPPORT: FEVER has slightly more REFUTE examples.

The threshold test (do this first, before looking at F1): Take 20 claims where you personally know the answer. Run your existing stance_b3 checkpoint. If it predicts NEUTRAL on > 12 of 20 obviously supported/refuted claims, the model is confidence-deflated and needs fine-tuning regardless of F1 scores.

Indic targets per language (IndicNLI):

- Hindi: accuracy > 0.80
- Tamil: > 0.77
- Telugu: > 0.74

- Kannada: > 0.73
- Malayalam: > 0.74

Full pipeline (end-to-end)

Dataset: FEVER test set + hand-curated 100 Indian claims (build this set yourself)
Script: `benchmarks/run_full_benchmark.py`

Metrics to report:

- Verdict accuracy on FEVER test (SUPPORT/REFUTE/NEI)
- Average pipeline latency per claim
- Evidence gathering success rate (% of claims where ⩾ 2 evidence pieces found)
- Per-stage drop-off: how many claims pass each checkpoint

PART 8 — SCORING SYSTEM REFERENCE

Evidence weight formula

```
EW(e) = RS(e) × CW(e) × TW(e)
```

Where:

- RS = Relevance Score from cross-encoder (0.0 – 1.0)
- CW = Credibility Weight from domain tier lookup (see table below)
- TW = Temporal Weight from article age (see table below)

Domain diversity multiplier (applied during MMR selection, before EW calculation):

- 1st evidence from a domain: × 1.00
- 2nd from same domain: × 0.70
- 3rd from same domain: × 0.50
- 4th+ from same domain: filtered out entirely (hard cap)

Credibility tier weights

Tier	CW	Category	Examples
T1	1.00	Government / Official scientific	pib.gov.in · rbi.org.in · nasa.gov · who.int

Tier	CW	Category	Examples
T2	0.95	International acclaimed orgs	un.org · unicef.org · worldbank.org · nature.com
T3	0.85	National wire services	pti.in · ani.in · reuters.com · apnews.com
T4	0.75	Reputable national news	thehindu.com · ndtv.com · indianexpress.com
T5	0.65	Regional news / fact-checkers	factly.in · altnews.in · boomlive.in · thequint.com
T6	0.40	General web / unknown	Unknown root domain · personal blogs
T7	0.15	Questionable / tabloid / satire	Known clickbait · parody news
T8	0.05	Social media / restricted	twitter.com · facebook.com · t.me

Wikipedia: treat as T6 (0.40) — useful background, not primary evidence for contested claims.

Temporal weights

Article age	TW	Exception
< 6 months	1.00	—
6–12 months	0.90	—
1–2 years	0.80	—
2–5 years	0.65	—
5–10 years	0.50	—
> 10 years / unknown	0.40	—
Any age	1.00	HISTORY and SCIENCE domains — temporal decay does not apply

Aggregation formula

```python
```

```
# Weighted sum across all evidence pieces
agg_support = sum(P_sup_i * EW_i for i in evidence) / sum(EW_i for i in evidence)
agg_refute  = sum(P_ref_i * EW_i for i in evidence) / sum(EW_i for i in evidence)
agg_neutral = sum(P_neu_i * EW_i for i in evidence) / sum(EW_i for i in evidence)

# Evidence count adjustment
count_multiplier = 0.75 if usable_evidence < 2 else (0.85 if usable_evidence <= 3 el

raw_confidence = max(agg_support, agg_refute, agg_neutral)
final_confidence = raw_confidence * count_multiplier

# Conflict detection
conflict_flag = (agg_support > 0.30 and agg_refute > 0.30)

# Raw verdict
raw_verdict = argmax(agg_support, agg_refute, agg_neutral)
```

Verdict label mapping

Raw verdict	Confidence	User-facing label
SUPPORT	≥ 0.75	Verified / True
SUPPORT	0.55 – 0.75	Likely True
SUPPORT	< 0.55	→ LLM Verifier
REFUTE	≥ 0.75	False / Misleading
REFUTE	0.55 – 0.75	Likely False
REFUTE	< 0.55	→ LLM Verifier
NEUTRAL	any	→ LLM Verifier always

Conflict flag adds: "Note: conflicting evidence exists" to any verdict.

PART 9 — SCORING TARGETS (ACADEMIC BENCHMARKS)

Checkpoint	Metric	Minimum acceptable	Target	Production ready
Checkability EN	Macro-F1	0.80	0.87	0.90+
Checkability Indic	Macro-F1	0.73	0.81	0.85+
Context EN	Accuracy (DBpedia)	0.83	0.90	0.95+
Context Indic	Macro-F1	0.74	0.81	0.86+
Relevance CE EN	Recall@5 (FEVER)	0.78	0.85	0.90+
Relevance CE Indic	Recall@5 (IndicQA)	0.70	0.78	0.84+
Stance EN	FEVER 3-way accuracy	0.83	0.89	0.92+
Stance EN	FEVER Macro-F1	0.80	0.87	0.91+
Stance Indic Hindi	IndicNLI accuracy	0.75	0.81	0.85+
Stance Indic Tamil	IndicNLI accuracy	0.72	0.78	0.83+
Stance Indic Telugu	IndicNLI accuracy	0.69	0.75	0.80+
Stance Indic Kannada	IndicNLI accuracy	0.68	0.74	0.79+
Stance Indic Malayalam	IndicNLI accuracy	0.69	0.75	0.80+
Full pipeline FEVER	Verdict accuracy	0.65	0.73	0.78+

PART 10 — KEY DECISIONS LOG (CHANGES FROM EARLIER VERSIONS)

Decision	Choice	Reason
Claim Type checkpoint	REMOVED	Checkability + Context already provide all routing signal
MMR placement	Evidence selection (Stage 6), NOT query generation	MMR is a diversity re-ranker, not a query generator

Decision	Choice	Reason
LLM verifier model	`llama-3.3-70b-versatile` on Groq	`openai/gpt-oss-20b` does not exist on Groq — would throw 404
Bi-encoder training	Off-shelf `multilingual-e5-small` for both EN + Indic	Off-shelf quality is sufficient; saves GPU quota for stance
Stance backbone	DeBERTa-v3-base (not large)	Academic resource constraint — base is 2% F1 lower but 3x cheaper
Checkability short-claim rule	REMOVED 6-word heuristic	Heuristic contradicts "less heuristic" goal; model handles short claims
LLM verifier timeout	20 seconds (not 40)	40s blows the pipeline SLA budget
Wikipedia credibility	Tier 6 (0.40)	Useful background, not authoritative for contested claims
PDF pipeline	Deferred — build after text + image stable	Adds complexity before core pipeline is validated
Image scope V1	Text-containing screenshots only	Charts, natural photos, handwriting are separate hard problems
Training single doc	This file	Single reference document for all decisions

---

Document version: 2.0 (updated) | Incorporates all feedback from analysis sessions AI4Bharat: github.com/AI4Bharat | CLEF CheckThat: checkthat.gitlab.io | FEVER: fever.ai